

# Neural Network and Back Propagation from Scratch

# Intro

# The Original Goal

The original goal came from pure curiosity:

*Build the smallest possible implementation of a neural network model.*

I wanted to understand how little code and machinery a working model actually needs.

At first, this sounded like a concrete engineering task.

## Why the Goal Is Ill-Posed

The goal described what to build, but not what should count as success.

- A network no better than a coin flip is still technically a model.
- If accuracy is 80%, should we continue to 81%, 82%, or 99%?
- Quality alone ignores training time and computational cost.
- A model that outperforms all state-of-the-art models combined after trillions of years of training is not a useful result.

A well-posed goal needs constraints

Define a target metric, a resource budget, and a stopping rule.

## Naive Algorithm: Brute-Force Selection

**Definitions:**  $W$  is a neural network,  $D$  is a dataset, and  $L(W, D)$  is a loss to minimize. The function `rand_net()` returns a random neural network.

```
best = rand_net()
for i = 1 to 1000:
    n1 = rand_net()
    if L(n1, D) < L(best, D):
        best = n1
```

Sample many random networks and keep the one with the lowest loss.

This may produce accuracy slightly above chance. But once it does, should we stop?

## Naive Algorithm №2: Random Local Search

```
best = rand_net()
step = 0.001
for i = 1 to 1000:
    g = rand_net() * step
    if L(best + g, D) < L(best, D):
        best = best + g
```

Instead of replacing the whole network, perturb its weights and accept only changes that reduce the loss.

### What changed?

The search procedure improved, but the objective did not become clearer.

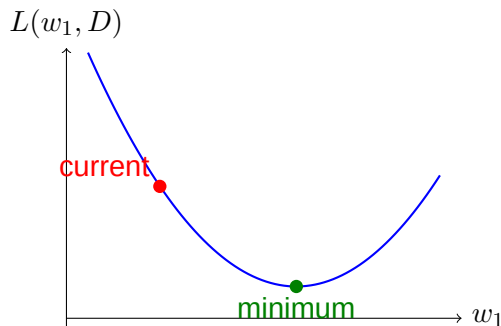
# Gradient Descent

# One Parameter, One Loss Curve

Consider a neural network with a single parameter  $w_1$ .  
For a fixed dataset  $D$ , its loss is now a function of one variable:

$$\ell(w_1) = L(w_1, D).$$

Training becomes a geometric problem: find the lowest point on this two-dimensional graph.



## From a Limit to a Numerical Derivative

Define  $f(w_1) = L(w_1, D)$ . The derivative is the limit of the slope between two increasingly close points:

### Exact derivative

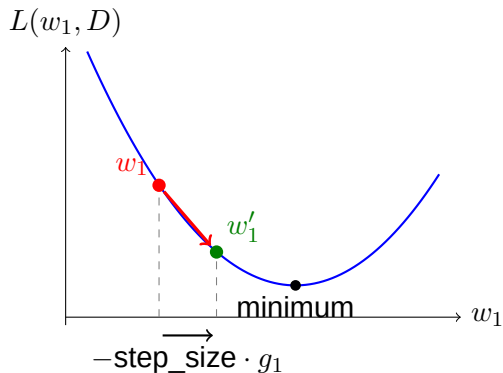
$$f'(w_1) = \lim_{h \rightarrow 0} \frac{f(w_1 + h) - f(w_1)}{h}.$$

### Finite difference

$$h = 1\text{e-}5$$
$$g_1 = \frac{L(w_1 + h, D) - L(w_1, D)}{h}.$$

For a small, finite  $h$ ,  $g_1 \approx f'(w_1)$ : it estimates how quickly the loss grows when  $w_1$  increases.

# One Gradient Descent Step



## Update rule

$$\text{step\_size} = 0.01$$

$$w'_1 = w_1 - \text{step\_size} \cdot g_1$$

$g_1$  points uphill, so the minus sign moves the parameter downhill.

Recompute the derivative at the new point and repeat.

# From One Parameter to Many

Let  $W = (w_1, \dots, w_n)$  and collect all partial derivatives in a vector  $G$ .

```
base_loss = L(W, D)
G = zeros(n)
for i = 1 to n:
    W_probe = W
    W_probe[i] += h
    G[i] = (L(W_probe, D) - base_loss) / h

W = W - step_size * G
```

\* We evaluate  $L$  independently for every parameter: once for each perturbed parameter, plus one shared evaluation of `base_loss`.

## Why Numerical Gradients Do Not Scale

With a sufficiently small  $h$ , finite differences can train the network. The problem is speed.

### Cost of one training step

For each of the  $N$  parameters, compute the loss  $L$  once:

$$\text{total work} \approx \underbrace{N}_{\text{number of parameters}} \times \underbrace{L}_{\text{one full loss evaluation}}.$$

Thus, one gradient step needs approximately  $N$  evaluations of  $L$ .

### The bottleneck

Networks that do anything non-trivial have a huge number of parameters. One training step therefore requires a huge number of evaluations of  $L$ .

# Optimizing Gradient Descent

# Stochastic Gradient Descent

Instead of computing the loss on the entire dataset at once, split the dataset into smaller batches:

$$D \longrightarrow B_1, B_2, \dots, B_k.$$

Compute the gradient on one batch, update the parameters, and continue with the next batch.

## Training loop

```
for batch in batches:  
    G = compute_gradient(L, W, batch)  
    W -= step_size * G
```

Here, `compute_gradient(L, W, batch)` means  $\nabla_W L(W, \text{batch})$ : the gradient of the batch loss with respect to  $W$ .

# Back Propagation

Here we develop an intuitive understanding of back propagation.

## Intuition

Back propagation is a way to compute gradients using the chain rule.

Understanding back propagation is critical to understanding how neural networks learn.

# Chain Rule

Recall the derivative of a function of one variable:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

For several inputs, keep the other inputs fixed:

$$f(x, y) = xy : \quad \frac{\partial f}{\partial x} = y, \quad \frac{\partial f}{\partial y} = x,$$

$$f(x, y) = x + y : \quad \frac{\partial f}{\partial x} = 1, \quad \frac{\partial f}{\partial y} = 1.$$

## Definition

The chain rule computes the derivative of a composition of two or more functions from their individual derivatives. If  $f(x) = g(h(x))$ , then

$$f'(x) = g'(h(x))h'(x).$$

## Compound Expressions with the Chain Rule

Consider a more complicated expression composed of multiple functions:

$$f(x, y, z) = (x + y)z.$$

Name the inner expression  $q = x + y$ . Then  $f = qz$ , and

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x},$$

$$\frac{\partial f}{\partial q} = z,$$

$$\frac{\partial f}{\partial x} = z \cdot 1 = z.$$

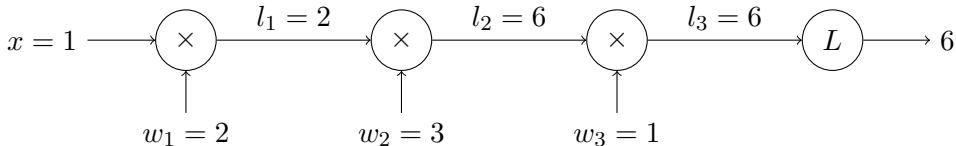
$$\frac{\partial q}{\partial x} = 1,$$

A trillion-parameter LLM, written as one formula at just 1 mm per parameter, would be about 2.6 Earth–Moon distances long.

## A Computation Graph

Back propagation works on any computation graph. A neural network is only one particular case.

For this example, use the simplest possible loss:  $L(l_3) = l_3$ .



First, evaluate the graph from left to right.

## Gradient for One Weight

$$g_2 = \frac{\partial L}{\partial w_2} = \frac{\partial l_2}{\partial w_2} \frac{\partial L}{\partial l_2}.$$

### Known from left to right

Since  $l_2 = l_1 w_2$ ,

$$\frac{\partial l_2}{\partial w_2} = l_1 = 2.$$

The value  $l_1$  was stored while evaluating the graph.

### Known from right to left

$$\frac{\partial L}{\partial l_2} = 1.$$

We computed this derivative earlier because back propagation moves from right to left.

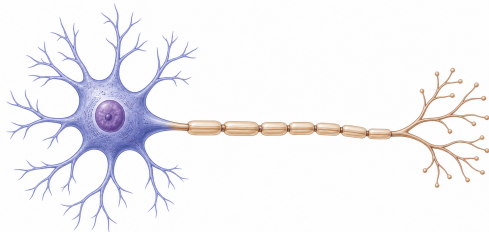
### Combine both values

$$g_2 = l_1 \frac{\partial L}{\partial l_2} = 2 \cdot 1 = 2.$$

# From Biological Neuron to Neural Network

# From Biology to Computation

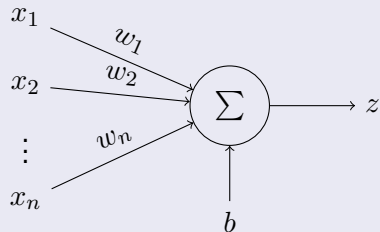
## Biological neuron



Receives signals through dendrites and sends one signal through the axon.

*The artificial neuron is an analogy, not a detailed simulation of a living cell.*

## Computational model



Computes one number from numeric inputs and weights.

# A Neuron as a Computation

The graph multiplies every input by its weight, adds the results, and adds a bias:

## Scalar form

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b.$$

Collect the inputs and weights into vectors:

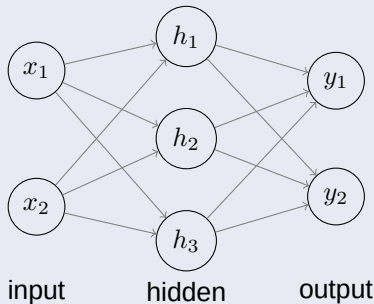
$$x = (x_1, \dots, x_n), \quad w = (w_1, \dots, w_n).$$

## Vector form

$$z = w^\top x + b.$$

## Same Network, Two Representations

Computation graph:  $2 \rightarrow 3 \rightarrow 2$



The same network as matrices

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \underbrace{\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix}}_{W_1 (3 \times 2)} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b_1$$

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \underbrace{\begin{bmatrix} v_{11} & v_{12} & v_{13} \\ v_{21} & v_{22} & v_{23} \end{bmatrix}}_{W_2 (2 \times 3)} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} + b_2$$

Each row of a matrix contains all incoming weights of one neuron.

## Without Nonlinearity, Depth Disappears

$$x \xrightarrow{W_1, b_1} h_1 \xrightarrow{W_2, b_2} h_2 \xrightarrow{W_3, b_3} y$$

$\equiv$

$$x \xrightarrow{W_*, b_*} y$$

$$\begin{aligned} y &= W_3(W_2(W_1x + b_1) + b_2) + b_3 \\ &= \underbrace{W_3W_2W_1}_{W_*}x + \underbrace{W_3W_2b_1 + W_3b_2 + b_3}_{b_*} \\ &= W_*x + b_*. \end{aligned}$$

# ReLU: A Simple Nonlinearity

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}'(x) = \begin{cases} 0, & x < 0, \\ 1, & x > 0. \end{cases}$$

## Why use ReLU?

- One `max` operation.
- Its derivative is just a binary mask.
- It works well in practice.

## Other choices

- Sigmoid or tanh
- Leaky ReLU
- GELU or SiLU

# One Layer in Pseudocode

One layer is a matrix–vector product, a bias, and a nonlinearity:

## Matrix-style pseudocode

```
def layer(x, W, b):  
    z = W @ x + b  
    output = maximum(z, 0)  
    return output
```

$$\underbrace{x}_{n \times 1} \xrightarrow{W \in \mathbb{R}^{m \times n}} \underbrace{Wx + b}_{m \times 1} \xrightarrow{\text{ReLU}} \underbrace{\max(0, Wx + b)}_{m \times 1}.$$

## Forward Pass Through Hidden Layers

Apply the same computation to every hidden layer and store each activation:

Simplified from `NN::forward_pass`

```
def forward(x, weights, biases):  
    activations = [x]  
  
    for layer in range(len(weights)):  
        z = weights[layer] @ x + biases[layer]  
        x = relu(z)  
        activations.append(x)  
  
    return activations
```

**Why store activations?** Back propagation reuses each layer's input to compute its weight gradient.

# Back Propagation Through Hidden Layers

Start with the loss derivative and visit the layers in reverse order:

Simplified from `NN::back_propagation`

```
def backpropagation(target, prediction, weights, activations):  
    d = prediction - target  
    for layer in reversed(range(len(weights))):  
        previous = activations[layer]  
        bias_grads[layer] += d  
        for row in range(len(d)):  
            for col in range(len(previous)):  
                weight_grads[layer][row][col] += d[row] * previous[col]  
        d = transpose(weights[layer]) @ d  
        d *= relu_derivative(previous)
```

$\text{ReLU}'(a)$  is 1 where  $a > 0$ , otherwise 0.

# One Mini-Batch Training Step

Simplified from `NN::train`

```
def train(batch_inputs, batch_targets):  
    for x, target in zip(batch_inputs, batch_targets):  
        activations = forward(x, weights, biases)  
        prediction = activations[-1]  
        backpropagation(target, prediction,  
                        weights, activations)  
    apply_grads()
```

Thank You!

# Thank you for your attention!

## Source code

[https://github.com/kvladsrc/gym/blob/main/neural\\_network/nn\\_lib/nn\\_lib.cc](https://github.com/kvladsrc/gym/blob/main/neural_network/nn_lib/nn_lib.cc)